

Lab 12

Pointers in C++

Objective

The Students will learn and apply the

- Computer memory and variables
- Pointers
- Pointers declaration
- Referencing operator
- Dereferencing operator

Introduction

In C++, every variable is stored in computer memory, and each memory location has an address. A pointer is a special variable that stores the address of another variable instead of storing a value directly.

Pointers help us work with memory directly and are very useful in programming.

Theory

1. Computer Memory and Variables

When we create a variable, the computer stores it in memory. Each variable has:

- A value
- A memory address

2. Pointers

A pointer is a variable that stores the address of another variable.

Example:

If $x = 10$, then a pointer can store the address of x .

3. Pointer Declaration

We declare a pointer using *.

Example:

```
int *ptr;
```

This means ptr is a pointer that can store the address of an integer.

4. Referencing Operator (&)

The & operator is used to get the address of a variable.

Example:

```
int x = 5;  
int *ptr = &x;
```

5. Dereferencing Operator (*)

The * operator is used to get the value stored at the address.

Example:

```
cout << *ptr;
```

This will print the value of x.

Example Program:

```
#include <iostream> // input/output ke liye library  
using namespace std; // std likhne ki zarurat na ho  
  
int main() {  
    int x = 10;           // ek integer variable x jisme value 10 hai  
    int *ptr;             // pointer variable jo integer ka address store karega  
  
    ptr = &x;             // ptr ko x ka address assign kiya ( & ka matlab address )  
  
    cout << "Value of x: " << x << endl;  
    // x ki actual value print hogi (10)  
  
    cout << "Address of x: " << &x << endl;  
    // x ka memory address print hoga  
  
    cout << "Pointer value (address): " << ptr << endl;  
    // ptr me jo address store hai (x ka address) wo print hoga  
  
    cout << "Value using pointer: " << *ptr << endl;  
    // pointer ko dereference karke x ki value print hogi (* ka matlab value nikalna)  
  
    return 0; // program successfully end  
}
```

Output:

Value of x: 10 Address of x: (some memory address) Pointer value (address): (same address) Value using pointer: 10

Conclusion:

In this lab, we learned that:

- Variables are stored in memory with addresses
- Pointers store addresses of variables
- & is used to get address
- * is used to get value from address

Pointers make programming more powerful and efficient.

Lab 12 Tasks:

1. Pointer Declaration and Initialization

- Declare an integer variable called num and initialize it with a value of your choice.
- Declare a pointer variable called ptr of type integer.
- Initialize the pointer variable to point to the address of the num variable.
- Display the value of num and the value pointed to by ptr using the de reference operator.

Code:

```

#include <iostream>
using namespace std;

int main() {

    int num = 10;
    // num ek integer variable hai aur isko 10 value di hai

    int *ptr;
    // ptr ek pointer variable hai jo integer ka address store karega

    ptr = &num;
    // ptr ko num ka address assign kiya (referencing operator & use hua)

    cout << "Value of num: " << num << endl;
    // num ki actual value print ho rahi hai

    cout << "Value pointed by ptr: " << *ptr << endl;
    // *ptr se num ki value mil rahi hai (dereferencing)

    return 0;
}

```

2. Pointer Arithmetic

- o Declare an integer array called numbers with some values of your choice.
- o Declare a pointer variable called ptr of type integer and initialize it to point to the first element of the num array.
- o Use pointer arithmetic to access and display the elements of the array using the pointer.

Code:

```

#include <iostream>
using namespace std;

int main() {

    int numbers[5] = {1, 2, 3, 4, 5};
    // numbers ek integer array hai jisme 5 values hain

    int *ptr;
    // ptr ek pointer hai jo integer type ka address store karega

    ptr = numbers;
    // pointer ko array ke first element ka address diya (numbers = &numbers[0])

    cout << "Array elements using pointer:" << endl;

    for(int i = 0; i < 5; i++) {
        cout << *(ptr + i) << endl;
        // pointer arithmetic: (ptr + i) se next elements access ho rahe hain
        // * lagane se value mil rahi hai (dereferencing)
    }

    return 0;
}

```